Propuesta y resultados del proyecto MiBici

Operaciones de aprendizaje automático · Equipo 14

Martha Alicia Villalobos Facundo
Milton Vera de la Torre
Víctor Villa Herrera

El proyecto analiza la demanda de viajes, la presión sobre la capacidad de las estaciones y sus patrones de uso. La ejecución integrada procesó 4,532,032 viajes de 2025 y registró los tres problemas en MLflow. P1 seleccionó el promedio histórico por su regla de validación; P2 seleccionó Random Forest; P3 seleccionó tres grupos. Los resultados permiten comparar alternativas con trazabilidad de datos y código, sin afirmar todavía una mejora operativa real.

Repositorio del proyecto en GitHub

Pregunta	Tarea y alcance	Usuario y decisión
P1 Cuántos viajes comenzarán	Regresión por estación y siguiente hora	Operador del sistema para planear flota
P2 Riesgo de pocas bicicletas o anclajes	Clasificación de presión aproximada por estación a tres horas	Equipo de rebalanceo para priorizar estaciones
P3 Patrones de uso y ubicación	Clustering de perfiles históricos por estación	Planeación urbana y expansión para estudiar necesidades de servicio

Datos y fuente

Indicador	Valor
Viajes y periodo	4,532,032; enero a diciembre de 2025
Fuentes	12 CSV mensuales y catálogo de estaciones; 393,104,868 bytes
Estaciones del catálogo	484; 480 con capacidad positiva
Controles de filas	0 inválidas, 0 duplicados y 0 identificadores contradictorios
Procesados	14 Parquet locales con filas, tamaño y SHA256

MiBici publica los archivos mensuales y la nomenclatura de estaciones (MiBici, s. f.). Los enlaces exactos están en references/sources.json y las cifras reales en data_manifest.json. El catálogo es de septiembre de 2026; su vigencia limita la interpretación histórica de 2025. No se identificó una licencia específica de redistribución en la página consultada; no se atribuye CC0 ni se publican viajes individuales. Se excluyen identificadores de usuarios y datos demográficos. Los tiempos se interpretan como hora local de Guadalajara.

ML Canvas y diseño de la solución

Bloque	Definición
Propuesta de valor	Apoyar la planeación de flota, el rebalanceo y el análisis de patrones de estaciones.
Usuarios	Operador del sistema, equipo de rebalanceo, planeación urbana y expansión.
Fuentes	Viajes mensuales de 2025 y catálogo oficial con capacidad y coordenadas.
Recolección	Descarga automática, normalización de columnas y codificación, validación y manifiesto SHA256.
Tareas	Regresión a una hora, clasificación de presión a tres horas y agrupamiento histórico.
Características	Calendario, rezagos, medias desplazadas, capacidad oficial y perfiles horarios de uso.
Salidas	Demanda esperada; puntuación y alerta de presión; grupo y perfil por estación.
Evaluación	MAE, RMSE y WAPE en P1; F2, recall, precisión y AP en P2; silhouette y estabilidad temporal en P3.
Uso	Apoyo a decisiones con revisión humana; análisis histórico del sistema.
Monitoreo	Calidad y cobertura de fuentes, cambios de capacidad, distribución de variables y desempeño posterior.
Criterio de éxito	Reconstruir datos y resultados, comparar baselines y candidatos en una partición temporal consistente.

Pipeline de aprendizaje automático

La ingesta convierte fuentes CSV en tablas operativas sin datos personales. Las salidas se contabilizan al inicio del viaje y las llegadas al finalizarlo. Se agregan eventos por estación y hora. features.py construye las variables históricas y, por separado, los objetivos futuros. Cada experimento entrena, selecciona con validación, evalúa en prueba y registra parámetros, métricas y artefactos.

En P1 se congela la población con salidas observadas en entrenamiento y presencia en el catálogo. En P2 se añade la capacidad y se construye la etiqueta de presión con eventos por minuto. En P3 se resumen perfiles por ventana, se ajusta el escalado en entrenamiento y se muestran después los grupos sobre coordenadas públicas. Las mismas funciones se reutilizan desde módulos; el notebook se reserva para exploración.

Las decisiones y sus supuestos se documentan en references/problem_definition.md, data_dictionary.md y variables_y_validacion.md. La unidad de análisis y el usuario se mantienen coherentes con cada tarea.

Preparación y arquitectura reproducible

Conjunto	Periodo de 2025	Uso
Entrenamiento	Enero a agosto	Ajustar modelos y transformaciones
Validación	Septiembre y octubre	Elegir modelo, umbral y número de grupos
Prueba	Noviembre y diciembre	Evaluar sin cambiar la selección

Se eligió un año completo para cubrir horarios, días y variación entre meses, sin una partición aleatoria que mezcle futuro y pasado. Los horizontes deben terminar dentro de su ventana. Los rezagos de 1, 24 y 168 horas y las medias de 24 y 168 horas se desplazan una hora. Las primeras 168 horas se excluyen de P1/P2 por historia insuficiente. En evaluación secuencial se utilizan únicamente horas anteriores ya cerradas.

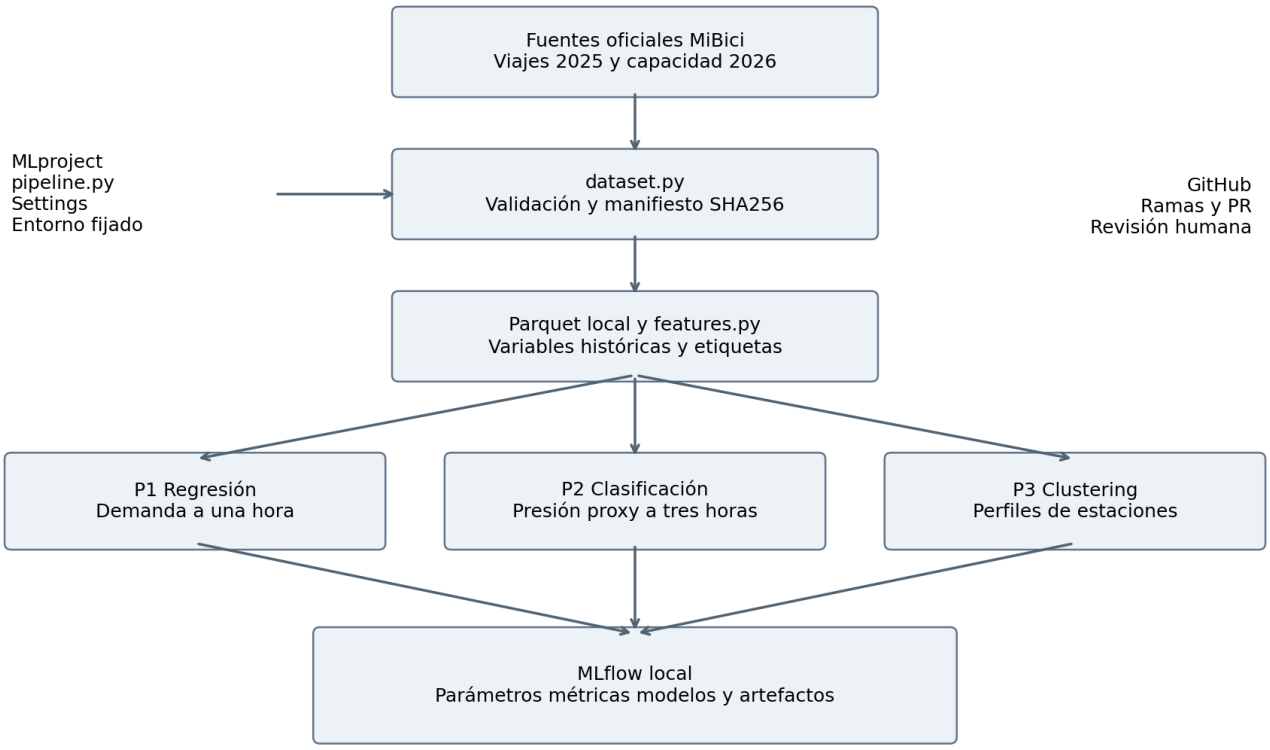


Figura 1. Flujo de fuentes, preparación, experimentos y registro.

Cookiecutter Data Science separa datos, módulos, notebooks, referencias y resultados (DrivenData, s. f.). Settings centraliza configuración; ExperimentTracker concentra trazabilidad; las clases RegressionExperiment, ClassificationExperiment y ClusteringExperiment agrupan responsabilidades. Pandas y PyArrow resuelven tablas y Parquet; scikit-learn aporta modelos y pipelines. MLflow Projects empaqueta parámetros y comandos, y Tracking conserva evidencia (MLflow, s. f.-b, s. f.-c). SQLite y artefactos locales permiten ejecutar el experimento sin servicios de nube.

El manifiesto conserva las huellas de fuentes y procesados. --verify rechaza cambios de versión antes de reemplazar los Parquet. Las dependencias están fijadas. Los datos, modelos serializados, almacenes de MLflow y credenciales quedan fuera de Git.

Demanda de viajes y definición del riesgo

P1 Regresión de demanda

Se evaluaron tres modelos sobre una cohorte de 369 estaciones. Quedaron 2,090,016 filas de entrenamiento, 540,216 de validación y 540,216 de prueba. Hubo 122 viajes fuera de la cohorte en validación y 433 en prueba, correspondientes a estaciones sin historia de entrenamiento; esta cobertura se registra por separado.

La referencia semanal repite la misma hora 168 horas antes. El promedio histórico se ajusta por estación, día y hora, con respaldo del promedio global de entrenamiento. HistGradientBoosting utiliza pérdida Poisson, 100 iteraciones, tasa 0.1, máximo 31 hojas y semilla 42. Sus 13 predictores incluyen historia de salidas/llegadas y calendario.

Modelo	MAE val	MAE prueba	RMSE prueba	WAPE prueba
Semanal	1.0333	0.9945	1.9142	75.11 %
Promedio histórico	0.8639	0.8776	1.5942	66.28 %
HistGradientBoosting	0.8582	0.8077	1.4314	61.00 %

El promedio fue seleccionado: su diferencia de MAE de validación con el candidato fue 0.005673, dentro de la tolerancia 0.01 que favorece al más sencillo. La prueba no modifica esa regla. El MAE mide viajes por estación y hora; RMSE penaliza errores grandes. WAPE divide el error absoluto total entre los viajes observados y no es porcentaje de aciertos. El promedio redujo el MAE de prueba en 11.76 % frente a la referencia semanal.

P2 Etiqueta de presión con capacidad

Se utiliza la capacidad oficial C de cada estación, con inventario inicial supuesto de 0.5 × C. Se acumulan llegadas menos salidas, minuto a minuto, durante las tres horas siguientes. La etiqueta vale 1 si el saldo cruza 0.1 × C o 0.9 × C en cualquier momento; se conserva la dirección bicicletas, anclajes o ambas. Los eventos futuros construyen la respuesta supervisada, nunca los predictores.

Ejemplo de la regla: con capacidad 20, ocho salidas netas reducen el saldo supuesto de 10 a 2 y activan presión; con capacidad 40, el saldo pasa de 20 a 12 y no la activa. La capacidad cambia la etiqueta; no se sustituye por el número de viajes. Este ejemplo no representa un caso observado.

La capacidad no positiva excluye la estación de P2. El inventario inicial y la aplicación del catálogo de 2026 a viajes de 2025 son supuestos explícitos. La etiqueta describe presión retrospectiva, no estaciones realmente vacías o llenas.

Resultados de clasificación de presión

Se comparan DummyClassifier prior y RandomForestClassifier de 200 árboles, profundidad 12, mínimo 10 ejemplos por hoja, pesos balanced y semilla 42. El ajuste utiliza una muestra uniforme de hasta 100,000 filas solo de entrenamiento; validación y prueba se evalúan completas. El umbral se elige por F2 de validación entre 0.10 y 0.90, con prioridad al umbral mayor en empate. F2 prioriza recall para no omitir casos de presión.

Modelo	F2 val	F2 prueba	Recall	Precisión	AP
Dummy prior	0.0000	0.0000	0.0000	0.0000	0.0704
Random Forest	0.6022	0.5630	0.8105	0.2534	0.3924

Random Forest fue seleccionado. En prueba recuperó 81.05 % de los positivos del escenario, con precisión de 25.34 % y F2 de 0.5630. La prevalencia fue 7.04 % y AP 0.3924. La baja precisión implica numerosas alertas que requerirían revisión; la puntuación no demuestra probabilidad calibrada de faltantes reales.

Matriz de confusión de prueba

Real / predicción	Sin presión	Con presión
Sin presión	413,051	91,171
Con presión	7,236	30,944

El artefacto evaluation/confusion_matrices.json conserva las matrices de validación y prueba dentro del run. La captura del punto 7 muestra el archivo abierto desde Artifacts, no una tabla recreada como sustituto de la interfaz.

Sensibilidad al inventario inicial

Inventario supuesto	Prevalencia	Coincidencia con 50 %
25 %	20.72 %	81.67 %
50 %	8.65 %	100.00 %
75 %	20.54 %	81.71 %

La prevalencia cambia de 8.65 % a cerca de 20.5–20.7 % al variar el inventario supuesto. Esta sensibilidad se calcula sobre el conjunto agregado de etiquetas, no solo sobre la prueba. La diferencia de denominadores explica que no coincida con la prevalencia 7.04 % de la evaluación final.

La validación integrada registra pequeñas diferencias numéricas frente a la referencia anterior de P2, con máximo absoluto 0.000707 entre las métricas comparadas. Se conserva Random Forest como selección y se informan las cifras de esta ejecución. No se afirma identidad numérica entre todas las ejecuciones.

Agrupamiento de estaciones y ubicación

Se construyen 99 variables por estación: perfiles horarios de salidas/llegadas en días laborables y fines de semana, proporción de fin de semana, volumen diario logarítmico y balance neto. StandardScaler y KMeans se ajustan solo con entrenamiento. Las coordenadas se añaden al mapa después del ajuste. Se exigen 28 días activos en entrenamiento y se comparan las 369 estaciones comunes entre ventanas.

k	Silhouette val	Silhouette prueba	ARI prueba	Davies Bouldin
2	0.0629	0.0565	0.8139	3.6524
3	0.0735	0.0564	0.8061	2.9941
4	0.0580	0.0020	0.5753	2.8419
5	0.0316	-0.0363	0.5422	2.9829
6	0.0075	-0.0205	0.4565	3.1575

Se seleccionó k=3 por silhouette de validación, mínimo cinco estaciones por grupo y tolerancia 0.01 a favor de menor k. La semilla es 42 y n_init=20. El silhouette de prueba 0.0564 indica separación débil, aunque el ARI temporal 0.8061 muestra estabilidad de asignación. No son métricas de accuracy ni justifican automáticamente zonas residenciales o comerciales.

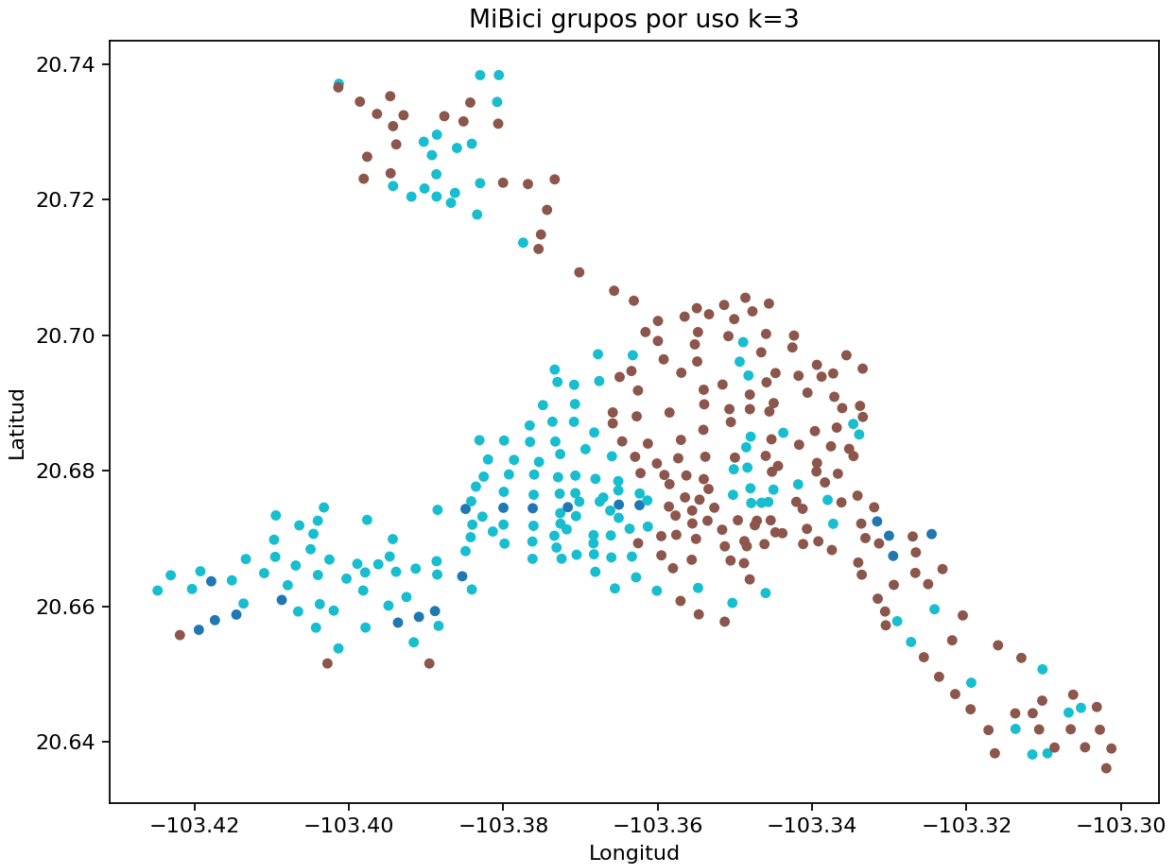


Figura 2. Ubicación de las 369 estaciones con grupos definidos por patrones de uso. Cada color representa un grupo; las coordenadas no participaron en el entrenamiento.

Reproducción buenas prácticas y aprendizajes

La ingesta desde fuentes públicas y la ejecución completa mediante MLflow Projects finalizaron con la versión de datos 97b8d9d898ca5f71ffaffc6ff86b100fbe371abfd5d67837d989e5f81bf9934f. El código ejecutado es 658e428a83101363e66dd4727675b51daac8b662, con git_dirty=false. El padre 10a63b549af148d3862f707678cf2105 y los diez hijos terminaron en FINISHED. Pasaron 18 pruebas y pip check no reportó incompatibilidades.

El README permite reconstruir el entorno con dependencias fijadas, descargar datos y ejecutar un comando parametrizado:

```
mlflow run . --env-manager local --experiment-name mibici-equipo14 -P question=all
```

MLproject también separa los entry points ingest y verify. Cambiar question o max_iter evita editar celdas. Cada modelo registra fechas, semilla, hiperparámetros, data_version, commit, pregunta, métricas con prefijo de conjunto y artefactos. Los resultados completos están en reports/reproduccion_local.json y la comprobación en reports/validacion_integrada.json. El ejemplo oficial de MLflow aporta el patrón de entry point y registro (MLflow, s. f.-a).

Git y organización

La rama principal exige PR aprobado por otra cuenta y aplica la protección a administradores. Las ramas separan cambios y los commits describen su propósito con feat, fix, docs o test. La integración conserva el historial original. Las responsabilidades se organizan en configuración e integración; ingesta y regresión; variables, clasificación y agrupamiento. El registro de GitHub permite revisar cambios y aprobaciones reales. Datos, modelos, credenciales y almacenes de MLflow quedan excluidos mediante .gitignore.

Aprendizajes

Refactorizar convirtió la exploración en un flujo que puede repetirse y revisarse por partes. Las funciones comparten transformaciones; las clases agrupan responsabilidades concretas sin añadir herencia innecesaria. La configuración central evita fechas y semillas dispersas (Real Python, s. f.).

La comparación con referencias sencillas fue decisiva. En P1, el candidato tuvo menor error, pero la regla previa eligió el promedio por una diferencia pequeña en validación. Cambiar esa regla después de ver la prueba habría debilitado la evaluación. En P2, la capacidad hace que los mismos movimientos tengan distinto significado; el recall debe leerse junto con el costo de falsas alertas. En P3, estabilidad no equivale a grupos bien separados.

Límites y mejora

La utilidad operativa necesita inventarios y rebalanceos históricos, demanda no atendida y seguimiento posterior. El catálogo es posterior a los viajes, los ceros no acreditan operación antes de una apertura y las estaciones nuevas requieren tratamiento separado. No se usan datos meteorológicos. Los resultados son una evaluación histórica; no se atribuye reducción real de costos, desabasto ni mejoras del servicio.

Referencias y evidencias de la entrega

DrivenData. (s. f.). *Cookiecutter Data Science*. https://cookiecutter-data-science.drivendata.org/

MiBici. (s. f.). *Datos abiertos*. https://mibici.net/es/datos-abiertos/

MLflow. (s. f.-a). *MLflow example* [Repositorio de código]. https://github.com/mlflow/mlflow-example

MLflow. (s. f.-b). *MLflow Projects*. https://mlflow.org/docs/latest/ml/projects/

MLflow. (s. f.-c). *MLflow Tracking*. https://mlflow.org/docs/latest/ml/tracking/

Real Python. (s. f.). *Object-oriented programming in Python*. https://realpython.com/python3-object-oriented-programming/

scikit-learn developers. (s. f.-a). *fbeta_score*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.fbeta_score.html

scikit-learn developers. (s. f.-b). *KMeans*. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html

Fuentes de los resultados

El informe usa el resumen de la ejecución integrada y el manifiesto de fuentes del repositorio. Conserva los resultados anteriores con sus commits de origen. La comparación de P1 utiliza la cohorte corregida y tres competidores; las cifras anteriores de un P1 de dos modelos corresponden a otra definición. Las capturas siguientes muestran los once runs, el detalle técnico de P2, una matriz registrada en Artifacts y la comparación de dos modelos P2.

Correspondencia de las capturas

Evidencia	Referencia
Lista de runs	Padre 10a63b549af148d3862f707678cf2105; 10 modelos hijos
Detalle y artefacto	P2 Random Forest a212dbfe379b420b83fbb03c300832fd
Comparación	P2 Random Forest frente a Dummy prior f49c0b91410a48009cd6a4c7d43849a3
Estructura GitHub	Rama revision/entrega-final, con módulos, referencias, reportes y configuración

Las capturas se tomaron desde la interfaz real. Se mostraron etiquetas técnicas y se ajustó el ancho de la columna de nombres para mejorar legibilidad. La captura de estructura recorta el área de carpetas y archivos. No se modificaron valores ni se fabricaron estados de ejecución.

Lista de runs del experimento

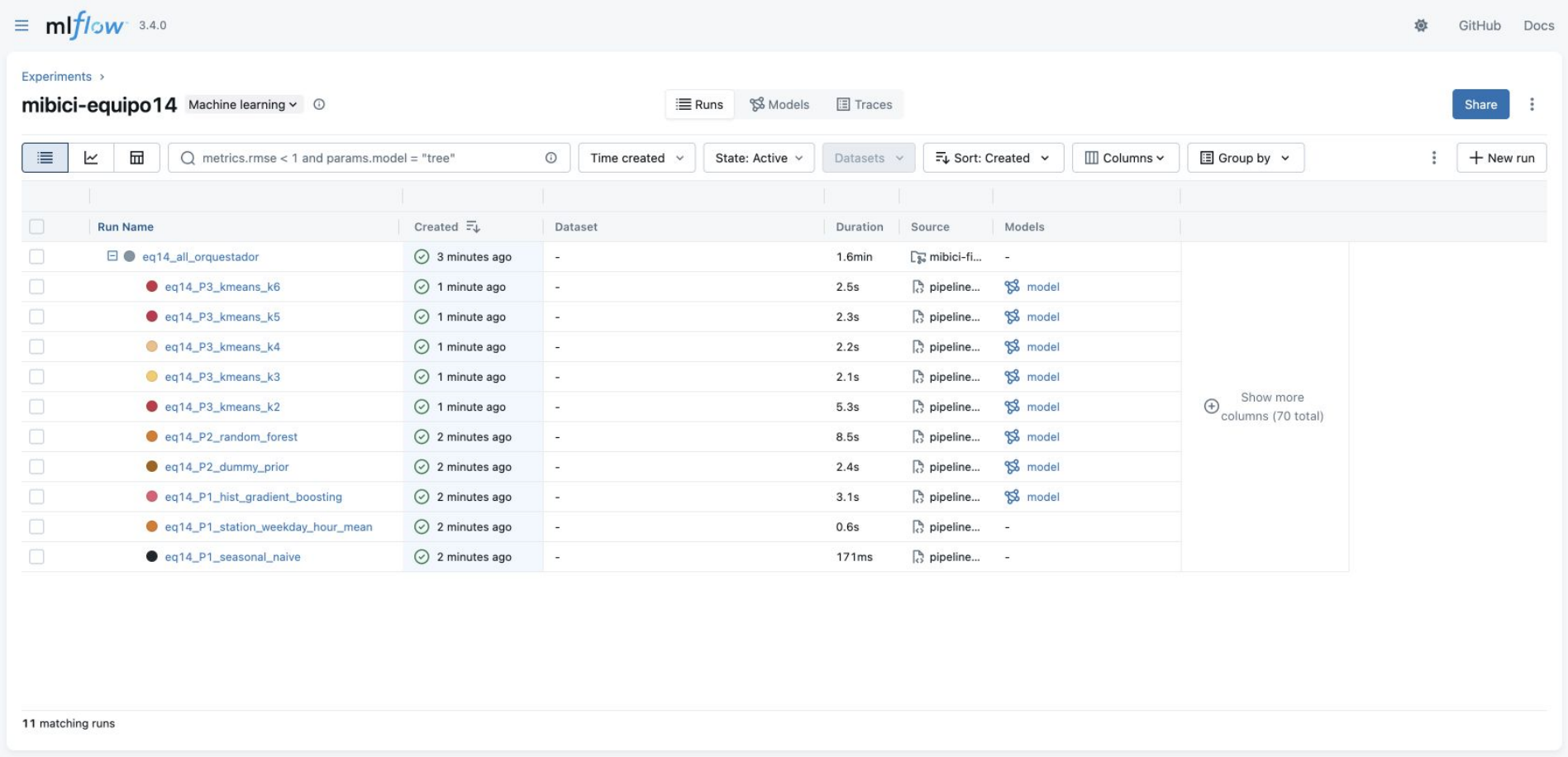


Figura 3. Un run padre y diez modelos hijos. Los indicadores de estado corresponden a ejecuciones terminadas.

Detalle de un run con parámetros métricas y etiquetas

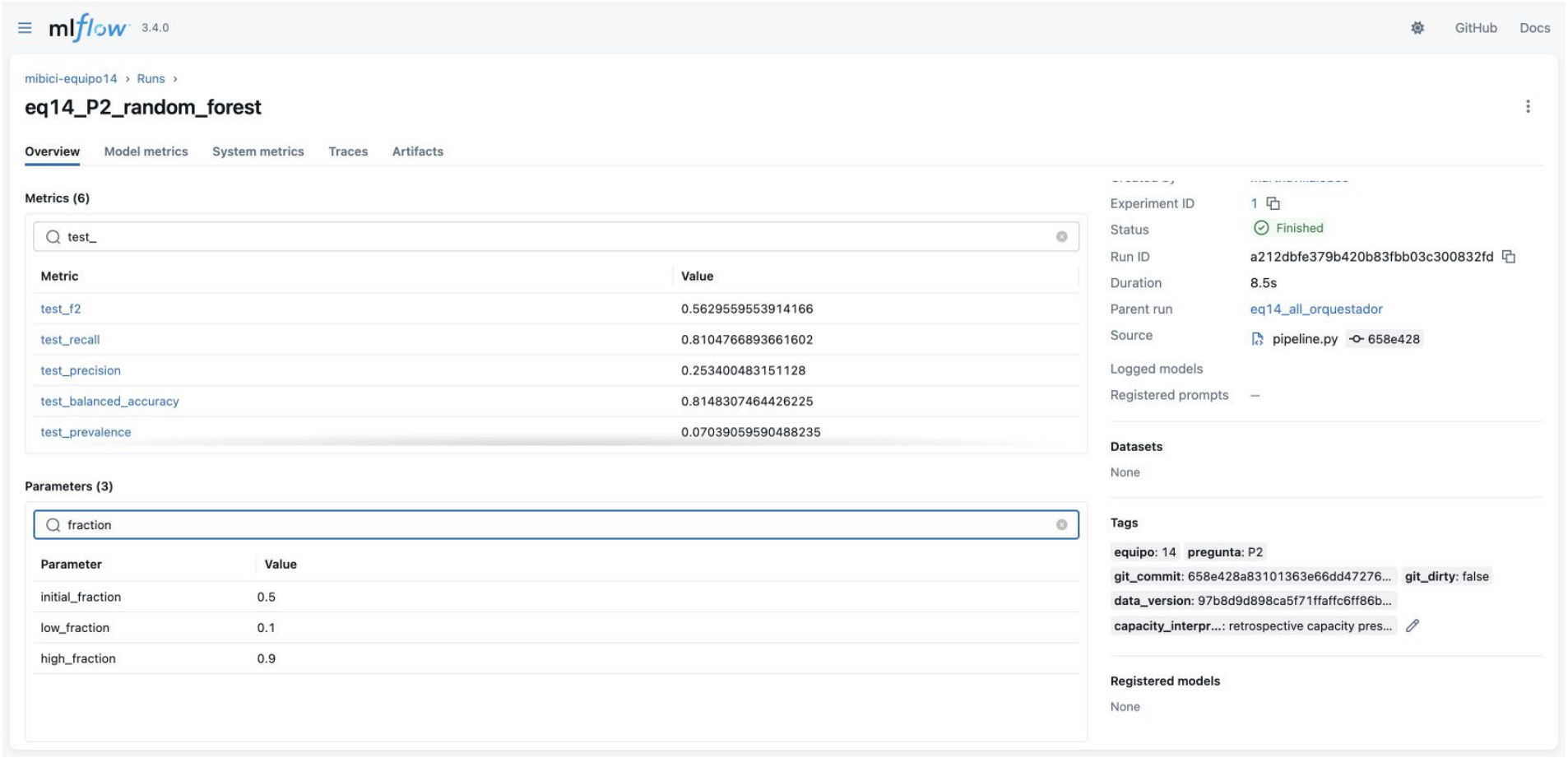


Figura 4. Run de P2 con métricas de prueba, parámetros del escenario de capacidad, pregunta, commit y data_version.

Artefacto registrado dentro de MLflow



Figura 5. Matrices de confusión de validación y prueba abiertas desde Artifacts del run de Random Forest.

Comparación de dos modelos de P2

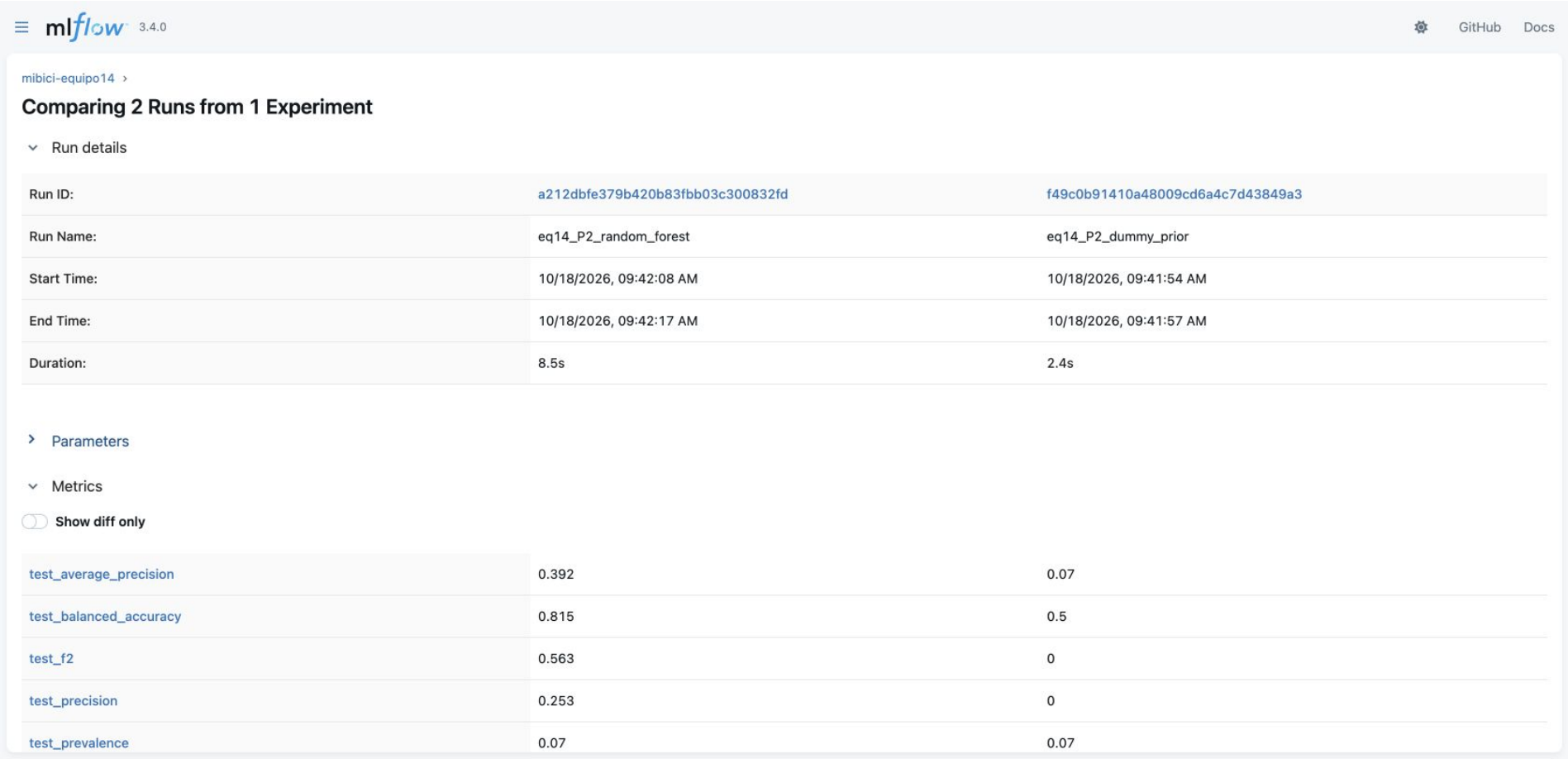
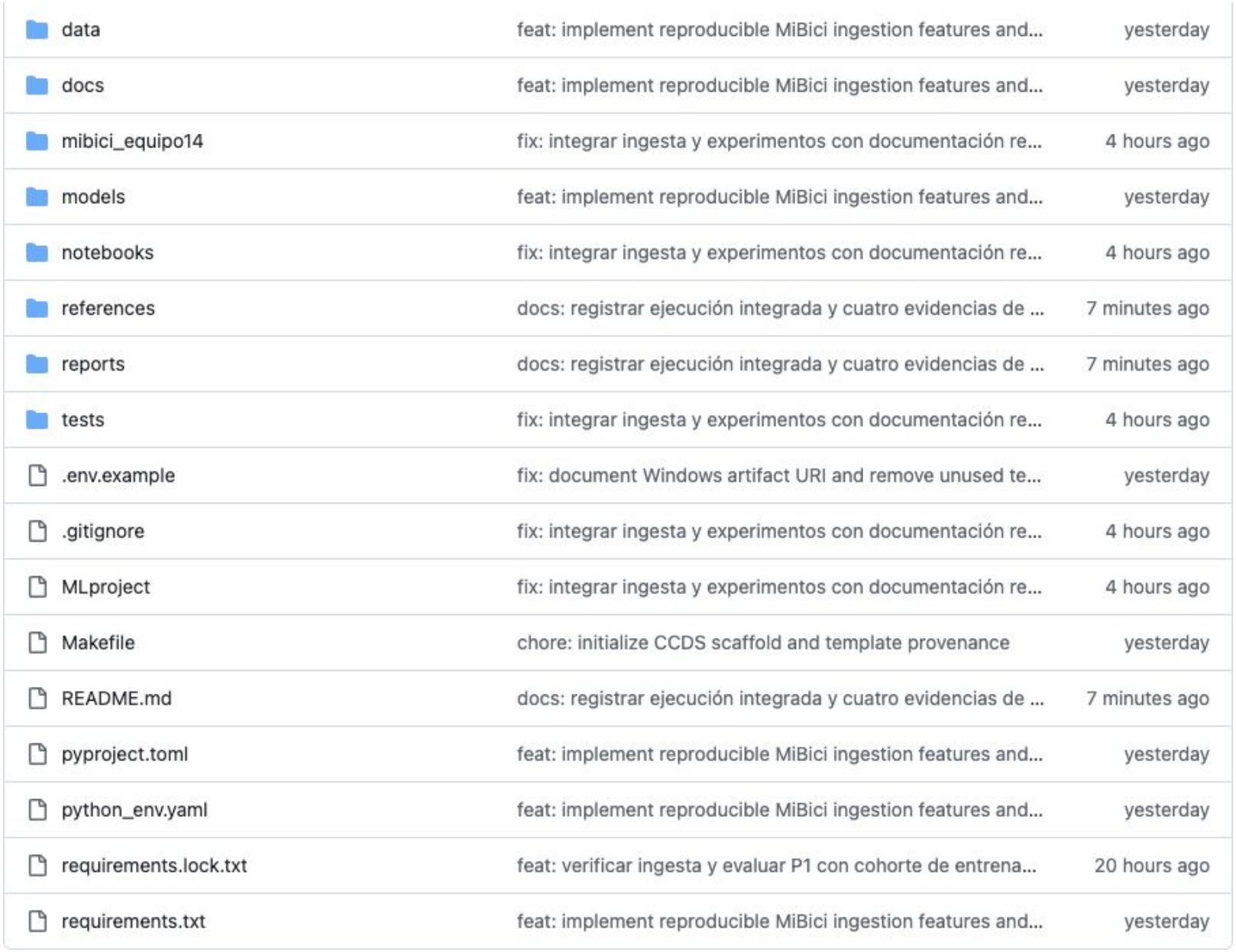


Figura 6. Los identificadores y nombres de Random Forest y Dummy prior se muestran junto a sus métricas de prueba.

Estructura publicada del repositorio



data	feat: implement reproducible MiBici ingestion features and...	yesterday
docs	feat: implement reproducible MiBici ingestion features and...	yesterday
mibici_equipo14	fix: integrar ingesta y experimentos con documentación re...	4 hours ago
models	feat: implement reproducible MiBici ingestion features and...	yesterday
notebooks	fix: integrar ingesta y experimentos con documentación re...	4 hours ago
references	docs: registrar ejecución integrada y cuatro evidencias de ...	7 minutes ago
reports	docs: registrar ejecución integrada y cuatro evidencias de ...	7 minutes ago
tests	fix: integrar ingesta y experimentos con documentación re...	4 hours ago
.env.example	fix: document Windows artifact URI and remove unused te...	yesterday
.gitignore	fix: integrar ingesta y experimentos con documentación re...	4 hours ago
MLproject	fix: integrar ingesta y experimentos con documentación re...	4 hours ago
Makefile	chore: initialize CCDS scaffold and template provenance	yesterday
README.md	docs: registrar ejecución integrada y cuatro evidencias de ...	7 minutes ago
pyproject.toml	feat: implement reproducible MiBici ingestion features and...	yesterday
python_env.yaml	feat: implement reproducible MiBici ingestion features and...	yesterday
requirements.lock.txt	feat: verificar ingesta y evaluar P1 con cohorte de entrena...	20 hours ago
requirements.txt	feat: implement reproducible MiBici ingestion features and...	yesterday

Figura 7. Carpetas y archivos de configuración en la rama revision/entrega-final del repositorio del equipo.